

ADVANCED TERRAFORM PRODUCTION

SENIOR ENGINEER NOTEBOOK

DESIGN. AUTOMATE.
OPERATE. SECURE.
AT SCALE.



PRODUCTION
ARCHITECTURE



STATE MANAGEMENT



MODULES & HCL



SECURITY &
GOVERNANCE



CI/CD & AUTOMATION



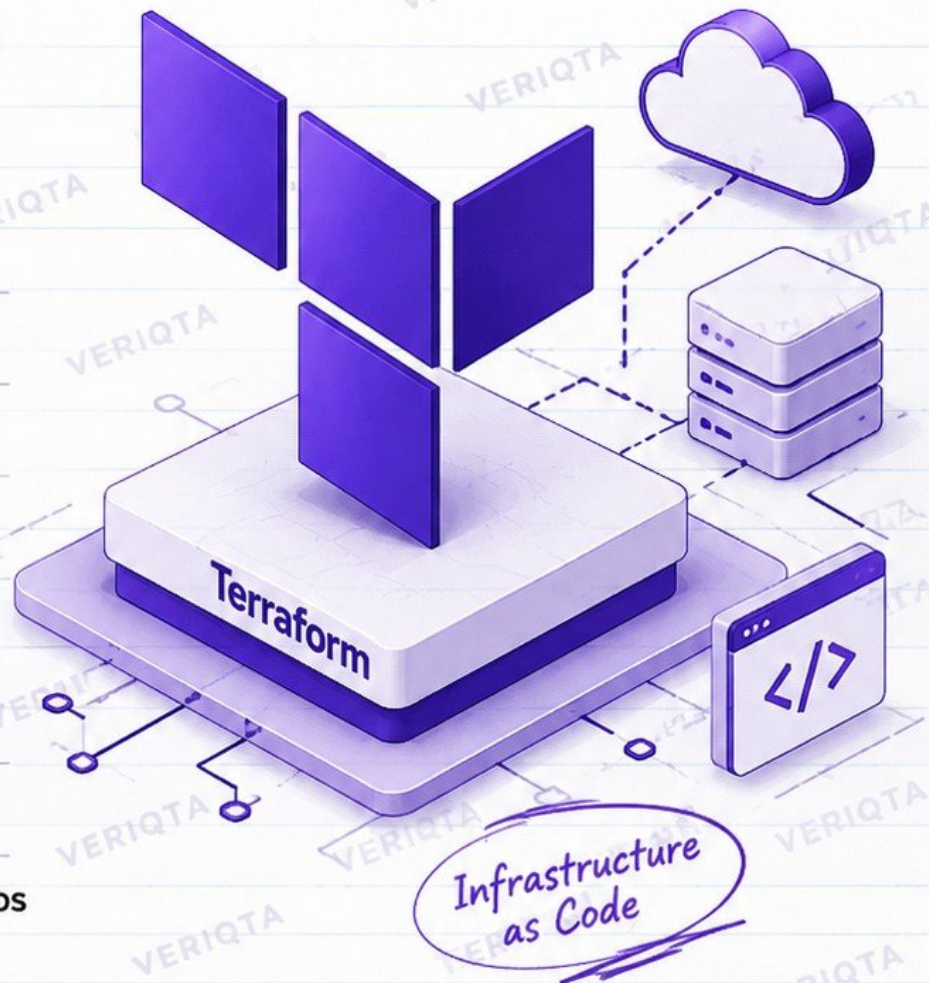
RELIABILITY & SCALE



TEAM OPERATIONS



REAL-WORLD SCENARIOS



CLEANER | SAFER | FASTER | STRONGER INFRASTRUCTURE



BY VERIQTA

01 PRODUCTION TERRAFORM ARCHITECTURE

DESIGNING TERRAFORM BOUNDARIES FOR REAL PRODUCTION SYSTEMS

1. WHY ARCHITECTURE MATTERS

- Prevents accidental changes in production
- Reduces blast radius
- Enables team collaboration
- Improves security and governance
- Supports multi-environment, multi-account and multi-region
- Makes infrastructure scalable and maintainable

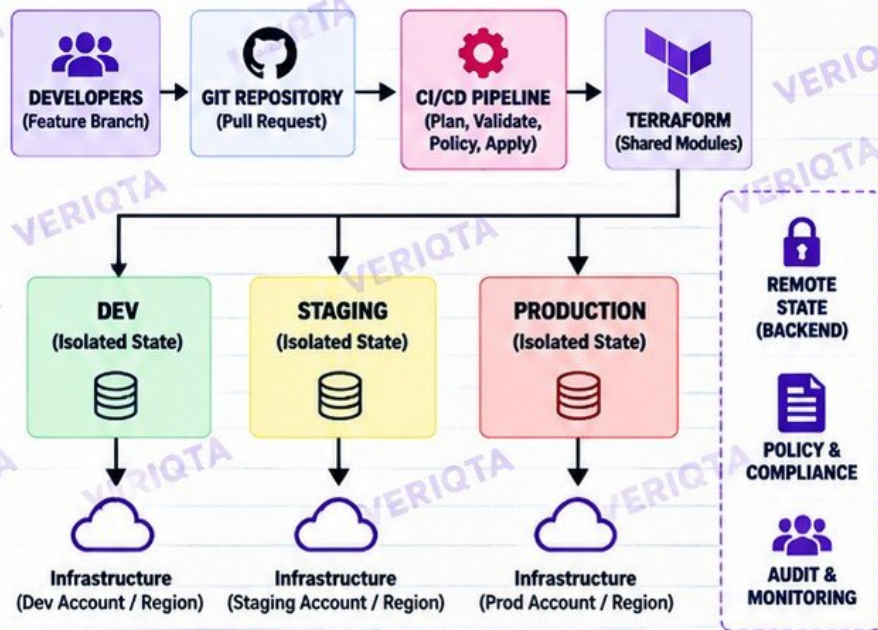
3. KEY DESIGN PRINCIPLES

- 1 SEPARATION OF CONCERNS**
Separate environments, accounts and teams.
- 2 SMALL, FOCUSED STATE**
Keep state files small and logical.
- 3 REUSABLE MODULES**
Standardize and avoid duplication.
- 4 AUTOMATION FIRST**
Use CI/CD, not manual applies.
- 5 SECURE BY DEFAULT**
Least privilege, encrypted state and policy controls.

4. COMMON ARCHITECTURE PATTERNS

SINGLE REPO	All environments in one repository (with clear structure)
MULTI-REPO	Separate repositories per environment or domain
WORKSPACES	Good for simple setups, limited isolation
DIRECTORY-BASED	Most common for production (more control and clarity)
MULTI-ACCOUNT	Stronger isolation and security
MODULE REGISTRY	Shared modules across teams

2. REFERENCE ARCHITECTURE



5. PRODUCTION CONSIDERATIONS

- ✓ Use remote state with locking (S3, Azure Blob, etc.)
- ✓ Separate state per environment and critical domain
- ✓ Use version-controlled modules
- ✓ Implement policy checks (OPA, Sentinel, etc.)
- ✓ Require code review for all changes
- ✓ Use short-lived cloud credentials (OIDC)
- ✓ Enable audit logging and monitoring
- ✓ Plan for disaster recovery (state backups)
- ✓ Document ownership and operational procedures

6. SENIOR ENGINEER TIP



"Your Terraform architecture is a product. Design it with the same care, scalability, security and maintainability as the systems you deploy."

STATE ARCHITECTURE, ISOLATION AND BLAST RADIUS

State topology, locking, isolation, recovery and failure domains

1. WHY REMOTE STATE IS ESSENTIAL

- Enables team collaboration
- Provides state locking
- Prevents concurrent conflicting changes
- Stores state securely and durably
- Supports automation (CI/CD)
- Allows proper access control and auditing
- Reduces risk of state loss or corruption

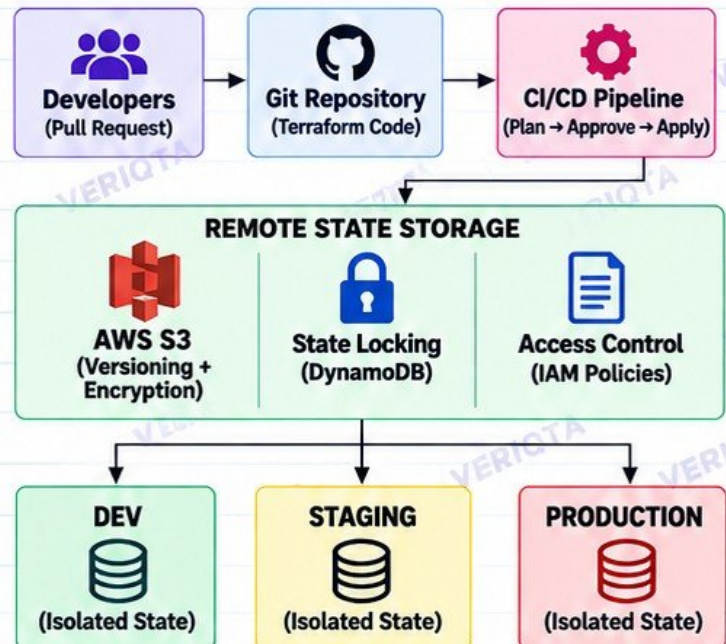
3. STATE TOPOLOGY OPTIONS

SINGLE STATE	Simple, but high blast radius. Suitable for small environments.
PER ENVIRONMENT	Separate state per environment (dev, staging, prod). Most common.
PER ACCOUNT	Separate state per cloud account. Improves isolation and security.
PER WORKLOAD/COMPONENT	State per service or component. Small blast radius and team independence.
HYBRID	Combination based on organization size and complexity.

5. FAILURE MODES AND IMPACT

FAILURE MODE	IMPACT
State file lost	Cannot reliably manage infrastructure
Lock not working	Concurrent changes can corrupt state
Single shared state	Large blast radius, harder recovery
No versioning	Difficult to restore previous state
Weak access control	Unauthorized or accidental modifications
State corruption	Inconsistent view of infrastructure

2. REFERENCE ARCHITECTURE



4. EXAMPLE: S3 REMOTE STATE CONFIGURATION

```

terraform {
  backend "s3" {
    bucket = "veriqta-terraform-state"
    key    = "prod/network/terraform.tfstate"
    region = "us-east-1"
    dynamodb_table = "veriqta-terraform-locks"
    encrypt  = true
  }
}
  
```

6. BEST PRACTICES

- ✓ Use remote state with encryption
- ✓ Enable state locking (e.g., DynamoDB)
- ✓ Separate state by environment or account
- ✓ Keep state files small and focused
- ✓ Enable versioning and backups
- ✓ Use least-privilege IAM policies
- ✓ Monitor and audit state access
- ✓ Test state recovery process regularly

7. SENIOR ENGINEER TIP

💡 "Design your state boundaries based on failure domains, not convenience. Smaller, isolated states reduce blast radius and make recovery faster and safer."

STATE SURGERY AND PRODUCTION RECOVERY

Safe state manipulation when Terraform and reality diverge

1. WHEN STATE SURGERY IS NEEDED

- Resource exists in cloud but not in state
- Resource in state but manually deleted
- Move resources between modules
- Rename or reorganize resources
- Recover from accidental changes
- Fix incorrect resource addresses
- Prepare for refactoring or migration

2. STATE SURGERY WORKFLOW



ALWAYS BACK UP YOUR STATE BEFORE MAKING ANY CHANGES.

3. COMMON STATE COMMANDS

COMMAND	PURPOSE
<code>terraform state list</code>	List all resources in state
<code>terraform state show <addr></code>	Show details of a resource
<code>terraform state rm <addr></code>	Remove a resource from state (does not delete real resource)
<code>terraform state mv <src> <dst></code>	Move resource to new address
<code>terraform import <addr> <id></code>	Import existing resource into state
<code>terraform state pull</code>	Download current state file
<code>terraform state push <file></code>	Upload a state file (use with care)

5. BEST PRACTICES

- ✓ Always back up state before changes
- ✓ Understand the impact before running commands
- ✓ Use `terraform plan` after state modifications
- ✓ Prefer `terraform import` over manual creation
- ✓ Use moved blocks in newer Terraform versions
- ✓ Keep state files secure and versioned
- ✓ Document all state operations for your team

6. RISKS AND HOW TO AVOID THEM

RISK	HOW TO AVOID
Accidental resource removal	Double-check addresses and run plan
State file corruption	Keep versioned backups (S3, etc.)
Inconsistent state and reality	Verify with plan and cloud console
Unauthorized changes	Restrict access and enable audit logs

4. EXAMPLE SCENARIOS



IMPORT EXISTING RESOURCE

```
terraform import aws_s3_bucket.logs my-bucket
```

Adds an existing S3 bucket to state.



MOVE RESOURCE

```
terraform state mv aws_instance.old aws_instance.new
```

Updates the resource address without recreating it.



REMOVE FROM STATE

```
terraform state rm aws_security_group.temp
```

Removes resource from state but keeps it in cloud.



INSPECT RESOURCE

```
terraform state show aws_db_instance.db
```

Shows full state details for a resource.

7. SENIOR ENGINEER TIP



"State is a record of what Terraform thinks exists. Treat it like a production database. Make changes deliberately, keep backups, and always verify."



Instagram
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Telegram
veriqta



Medium
veriqta

**BUILD
A BETTER TOMORROW**

PRODUCTION MODULE ARCHITECTURE

Designing reusable modules without creating abstraction debt

1. WHY MODULES MATTER

- ✓ Promote reuse and consistency
- ✓ Reduce duplication and manual effort
- ✓ Standardize best practices
- ✓ Improve maintainability
- ✓ Enable faster, safer deployments
- ✓ Support multi-environment and multi-account setups

3. MODULE DESIGN PRINCIPLES

- 1 CLEAR INPUT / OUTPUT INTERFACES**
Well-defined variables and outputs.
- 2 SINGLE RESPONSIBILITY**
One logical component per module.
- 3 REUSABLE AND COMPOSABLE**
Work across environments and accounts.
- 4 OPINIONATED BUT FLEXIBLE**
Provide sensible defaults with override options.
- 5 VERSIONED**
Use semantic versioning and a module registry.
- 6 WELL DOCUMENTED**
Include README, examples and usage guidance.

5. COMMON ANTI-PATTERNS

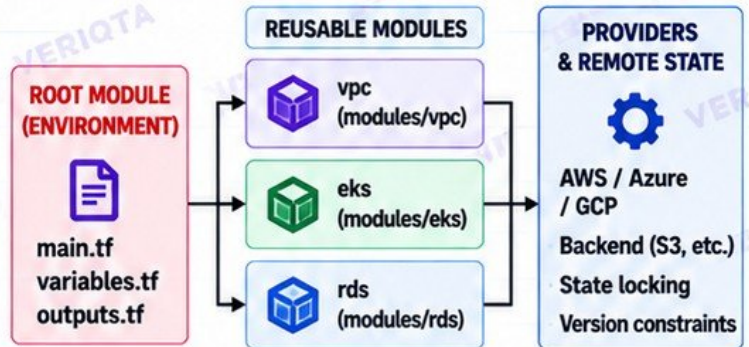
- ✗ Monolithic modules that do everything
- ✗ Modules with too many variables
- ✗ Copying and modifying modules per environment
- ✗ No versioning or using random branches
- ✗ Tight coupling between modules
- ✗ Modules without documentation or examples

7. SENIOR ENGINEER TIP



"A good module hides complexity.
A great module gives control without
making the user think about the implementation."

2. MODULE ARCHITECTURE



4. EXAMPLE: USING A MODULE

```

module "vpc" {
  source = "git::https://github.com/veriqta/modules/vpc.git"
  version = "1.2.0"

  name       = "prod-vpc"
  cidr_block = "10.0.0.0/16"
  azs        = ["us-east-1a", "us-east-1b", "us-east-1c"]
  enable_nat = true
  tags = {
    Environment = "prod"
    Project      = "veriqta"
  }
}
  
```

6. MODULE REPOSITORY STRUCTURE

```

terraform-modules/
├── vpc/
├── eks/
├── rds/
├── iam/
├── networking/
├── security/
├── README.md
└── examples/
  
```

- ✓ Logical structure
- ✓ Each module self-contained
- ✓ Examples for real usage
- ✓ Clear version tags
- ✓ Automated testing (optional)
- ✓ Publish to private or public registry

8. KEY TAKEAWAY



Design modules for long-term use, not just for today's need. Reusable, well-structured modules reduce risk, speed up delivery, and keep your infrastructure maintainable at scale.



ADVANCED HCL AND DYNAMIC INFRASTRUCTURE

Building expressive infrastructure logic without making configuration unmaintainable

1. WHY ADVANCED HCL MATTERS

- ✓ Build flexible and reusable infrastructure
- ✓ Reduce duplication and manual work
- ✓ Adapt to different environments and inputs
- ✓ Handle complex real-world requirements
- ✓ Keep configurations DRY and maintainable

2. CORE LANGUAGE FEATURES

FEATURE	USE CASE
Variables & types	Strongly typed, validated inputs
Locals	Simplify complex expressions
Conditionals	Create resources based on conditions
for_each	Create multiple resources from a map/set
count	Create multiple resources from a number
Dynamic blocks	Generate nested blocks dynamically
Functions	Transform and manipulate data
Object and list types	Work with structured data
try() and can()	Handle optional or unknown values
merge() and flatten()	Combine and reshape data

3. EXAMPLE: for_each WITH A MAP

```
resource "aws_instance" "app" {  
  for_each = var.instances  
  ami      = each.value.ami  
  instance_type = each.value.type  
  tags = {  
    Name      = each.key  
    Environment = var.environment  
  }  
}
```

Example input (tfvars)

```
instances = {  
  web = {  
    ami = "ami-123"  
    type = "t3.micro"  
  }  
  api = {  
    ami = "ami-456"  
    type = "t3.small"  
  }  
}
```

4. EXAMPLE: DYNAMIC BLOCKS

```
resource "aws_security_group" "app" {  
  name = "app-sg"  
  
  dynamic "ingress" {  
    for_each = var.ingress_rules  
    content {  
      from_port = ingress.value.from  
      to_port   = ingress.value.to  
      protocol  = ingress.value.protocol  
      cidr_blocks = ingress.value.cidr  
    }  
  }  
}
```

Example input

```
ingress_rules = [  
  {  
    from = 80  
    to   = 80  
    protocol = "tcp"  
    cidr = "0.0.0.0/0"  
  },  
  {  
    from = 443  
    to   = 443  
    protocol = "tcp"  
    cidr = "0.0.0.0/0"  
  }  
]
```

5. USEFUL FUNCTIONS

FUNCTION	PURPOSE
merge()	Combine multiple maps
flatten()	Flatten nested lists
lookup()	Get a value with a default
element()	Get an item from a list
length()	Get the size of a list or map
try()	Return first successful expression
can()	Check if an expression is valid
tomap() / tolist()	Convert types
format()	Create formatted strings

6. BEST PRACTICES

- ✓ Keep logic simple and readable
- ✓ Use locals for complex expressions
- ✓ Use meaningful variable names
- ✓ Validate inputs with type constraints
- ✓ Avoid overly complex nested logic
- ✓ Document non-trivial expressions
- ✓ Prefer for_each over count for stable keys
- ✓ Keep modules flexible but not overly generic
- ✓ Test changes with terraform plan

7. COMMON PITFALLS

- ✗ Overusing complex expressions
- ✗ Making code hard to read and maintain
- ✗ Inconsistent data types
- ✗ Not validating inputs
- ✗ Creating too much abstraction
- ✗ Using count when order can change

8. SENIOR ENGINEER TIP



"HCL is powerful, but clarity beats cleverness. Write configurations that your team can maintain in two years, not just ones that work today."

RESOURCE LIFECYCLE AND DEPENDENCY CONTROL

Understanding Terraform's dependency graph and controlling destructive behavior

1. WHY IT MATTERS

- ✓ Controls how resources are created, updated and destroyed
- ✓ Prevents accidental downtime and data loss
- ✓ Ensures correct resource ordering
- ✓ Helps manage changes safely in production
- ✓ Gives you fine-grained control over destructive behavior
- ✓ Reduces dependency and race condition issues

2. HOW TERRAFORM BUILDS THE DEPENDENCY GRAPH

- Terraform analyzes references between resources.
- It builds a dependency graph to determine the correct order.
- Resources are created, updated or destroyed based on this graph.
- Implicit dependencies: created automatically from references (e.g., using resource IDs).
- Explicit dependencies: defined using `depends_on` when needed.

EXAMPLE DEPENDENCY GRAPH



Terraform creates resources in the correct order based on dependencies.

3. LIFECYCLE META-ARGUMENTS

ARGUMENT	PURPOSE
<code>create_before_destroy</code>	Creates new resource before destroying the old one (avoids downtime).
<code>prevent_destroy</code>	Prevents accidental deletion of a resource.
<code>ignore_changes</code>	Ignores changes to specific attributes.
<code>replace_triggered_by</code>	Forces replacement when specified resources change.
<code>precondition</code>	Ensures a condition is met before creating a resource.
<code>postcondition</code>	Validates a condition after resource creation.

4. EXAMPLE: LIFECYCLE CONFIGURATION

```

resource "aws_instance" "app" {
  ami           = "ami-12345678"
  instance_type = "t3.micro"
  tags = {
    Name = "app-server"
  }
  lifecycle {
    create_before_destroy = true
    prevent_destroy       = true
    ignore_changes        = ["tags"]
    replace_triggered_by  = [aws_launch_template.app]
  }
}
  
```

What this does:

- ✓ Creates new instance before destroying old one
- ✓ Prevents accidental deletion
- ✓ Ignores tag changes
- ✓ Replaces instance when launch template changes

5. EXPLICIT DEPENDENCIES

```

resource "aws_instance" "app" {
  ami           = "ami-12345678"
  instance_type = "t3.micro"
  depends_on = [aws_security_group.app]
}
  
```

Use `depends_on` when Terraform cannot automatically determine the dependency (e.g., resources created with `local-exec`, `null_resource` or complex logic).

6. COMMON SCENARIOS

SCENARIO	SOLUTION
Avoid downtime on updates	Use <code>create_before_destroy</code>
Protect critical resources	Use <code>prevent_destroy</code>
Ignore changes to specific fields	Use <code>ignore_changes</code>
Force replacement on external change	Use <code>replace_triggered_by</code>
Validate inputs before creation	Use <code>precondition</code>
Validate results after creation	Use <code>postcondition</code>
Ensure correct ordering	Use <code>depends_on</code>

7. BEST PRACTICES

- ✓ Understand the dependency graph before making changes
- ✓ Use lifecycle rules for critical resources
- ✓ Avoid unnecessary explicit dependencies
- ✓ Test changes in non-production environments
- ✓ Document why lifecycle rules are used
- ✓ Review plans carefully for destructive actions

8. SENIOR ENGINEER TIP



"Terraform gives you power to change infrastructure. Lifecycle rules and dependency control help you use that power safely."

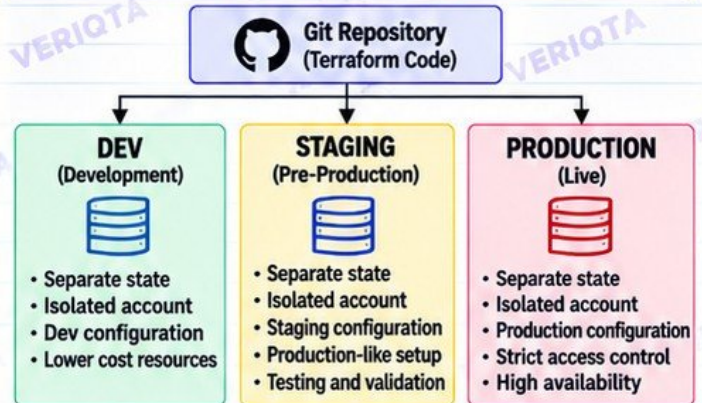
MULTI-ENVIRONMENT ARCHITECTURE

Dev, staging and production isolation without configuration duplication

1. WHY MULTI-ENVIRONMENT ARCHITECTURE

- ✓ Isolates environments (dev, staging, prod)
- ✓ Prevents accidental changes in production
- ✓ Enables safe testing and validation
- ✓ Supports team collaboration
- ✓ Allows reuse of the same code with different configurations
- ✓ Improves security, cost control and compliance

2. ENVIRONMENT ARCHITECTURE OVERVIEW



3. COMMON IMPLEMENTATION APPROACHES

APPROACH	DESCRIPTION	USE CASE
Separate Directories	Each environment has its own folder (dev, staging, prod).	Most common and simple to manage.
Workspaces	Single codebase with Terraform workspaces.	Useful for smaller setups.
Separate Repositories	Different repos per environment.	Larger organizations with strict isolation requirements.
Module + Variables	Reusable modules with environment-specific variables.	Best for consistency and scalability.

4. EXAMPLE: DIRECTORY STRUCTURE

```

terraform-project/
├── modules/
│   ├── vpc/
│   ├── eks/
│   └── iam/
├── environments/
│   ├── dev/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── terraform.tfvars
│   ├── staging/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── terraform.tfvars
│   └── prod/
│       ├── main.tf
│       ├── variables.tf
│       └── terraform.tfvars
  
```

KEY POINTS

- ✓ Same modules used across all environments
- ✓ Different variable values per environment
- ✓ Separate state files per environment
- ✓ Environment-specific backend configuration
- ✓ Keep sensitive values out of code (use secrets)

5. EXAMPLE: ENVIRONMENT VARIABLES

```

# environments/dev/terraform.tfvars
environment = "dev"
region     = "us-east-1"
instance_type = "t3.micro"
vpc_cidr    = "10.0.0.0/16"

# environments/prod/terraform.tfvars
environment = "prod"
region     = "us-east-1"
instance_type = "t3.large"
vpc_cidr    = "10.1.0.0/16"
  
```

GOOD PRACTICES

- ✓ Use the same codebase
- ✓ Override values per environment
- ✓ Keep sensitive data in environment variables or secret managers
- ✓ Validate changes with terraform plan
- ✓ Use naming conventions

7. COMMON MISTAKES

- ✗ Using the same state for multiple environments
- ✗ Hardcoding environment-specific values
- ✗ No separation between dev and production
- ✗ Accidental deployment to the wrong environment
- ✗ Not using remote state or locking

6. STATE ISOLATION STRATEGY

COMPONENT	RECOMMENDATION
Backend	Use remote backend (S3, Azure Blob, etc.)
State per environment	Separate state file for dev, staging, prod
State locking	Enable locking (e.g., DynamoDB, Azure)
Access control	Restrict access per environment
Encryption	Enable encryption at rest and in transit
Naming	Use clear and consistent naming

8. SENIOR ENGINEER TIP



"Design your environment strategy early. Isolation is cheaper than fixing mistakes in production."

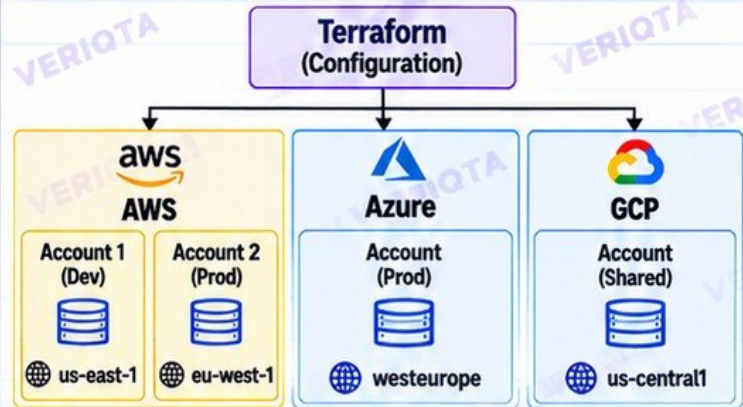
MULTI-ACCOUNT, MULTI-REGION AND MULTI-PROVIDER DESIGN

Provider boundaries, credentials, regions, accounts and failure isolation

1. WHY IT MATTERS

- ✓ Separates workloads across accounts and regions
- ✓ Improves security and compliance
- ✓ Reduces blast radius and operational risk
- ✓ Enables scalable global infrastructure
- ✓ Supports multiple cloud providers
- ✓ Gives you flexibility for disaster recovery
- ✓ Prevents accidental changes in the wrong account/region

2. ARCHITECTURE OVERVIEW



Multi-provider. Multi-account. Multi-region. Isolated and secure.

3. PROVIDER CONFIGURATION EXAMPLE

AWS provider for different accounts and regions

```
provider "aws" {
  alias   = "prod"
  region  = "us-east-1"
  profile = "prod-account"
}
```

```
provider "aws" {
  alias   = "dr"
  region  = "eu-west-1"
  profile = "dr-account"
}
```

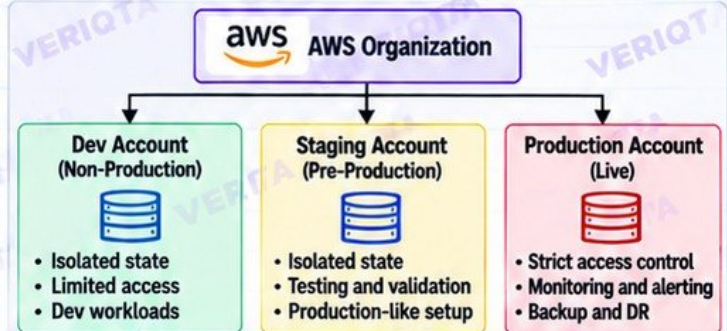
```
# Azure provider
provider "azurerm" {
  features {}
  subscription_id = "xxxx-xxxx-xxxx"
}
```

```
# Google Cloud provider
provider "google" {
  project = "my-gcp-project"
  region  = "us-central1"
}
```

KEY POINTS

- ✓ Use provider aliases for multiple accounts/regions
- ✓ Keep credentials secure (use profiles, service accounts or OIDC)
- ✓ Define explicit regions
- ✓ Use least privilege access
- ✓ Separate state per account/region

4. MULTI-ACCOUNT STRUCTURE (AWS EXAMPLE)



5. REGION STRATEGY

USE CASE	REGION STRATEGY
Low latency for users	Deploy in regions close to users
High availability	Use multiple regions
Disaster recovery	Keep replicated infrastructure
Compliance requirements	Use specific regions (e.g., EU)
Cost optimization	Choose cost-effective regions
Global applications	Use a multi-region architecture

6. BEST PRACTICES

- ✓ Use separate state for each account and region
- ✓ Use provider aliases and clear naming
- ✓ Manage credentials securely (no hardcoding)
- ✓ Implement least privilege access
- ✓ Use consistent tagging and naming conventions
- ✓ Design for failure isolation and disaster recovery
- ✓ Document your account and region strategy

7. COMMON MISTAKES

- ✗ Using the same state across accounts
- ✗ Hardcoding credentials
- ✗ Deploying to the wrong account/region
- ✗ Not isolating environments properly
- ✗ Overly complex provider configurations
- ✗ Ignoring region-specific limitations
- ✗ Not planning for failure and recovery

8. SENIOR ENGINEER TIP



"Design for isolation from day one. A good multi-account and multi-region strategy saves you from a bigger incident later."

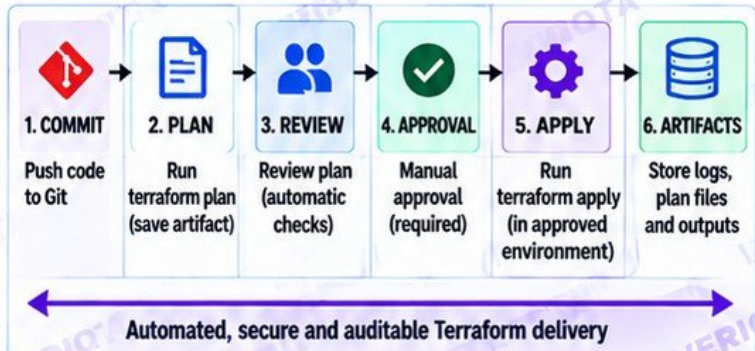
TERRAFORM CI/CD PRODUCTION PIPELINE

Safe plan, review, approval, apply and artifact workflows

1. WHY CI/CD FOR TERRAFORM

- ✓ Prevents manual and risky changes
- ✓ Enforces consistent and repeatable workflows
- ✓ Adds plan, review and approval before apply
- ✓ Improves security, auditability and compliance
- ✓ Stores artifacts (plan files, logs, state info)
- ✓ Reduces human error
- ✓ Supports multi-environment deployments
- ✓ Gives full visibility and traceability

2. HIGH-LEVEL PIPELINE FLOW



3. EXAMPLE: GITHUB ACTIONS PIPELINE

name: Terraform CI/CD
on:

push:
branches: ["main"]
pull_request:
branches: ["main"]

jobs:
plan:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- name: Setup Terraform
uses: hashicorp/setup-terraform@v3
- name: Terraform Init
run: terraform init
- name: Terraform Plan
run: terraform plan -out=tfplan
- name: Upload Plan Artifact
uses: actions/upload-artifact@v4
with:
name: tfplan
path: tfplan

KEY POINTS

- ✓ Run plan on pull requests
- ✓ Save plan file as artifact
- ✓ Require manual approval before apply
- ✓ Use environment protection rules
- ✓ Store logs and artifacts for audit
- ✓ Repeat for each environment (dev, staging, prod)

4. ENVIRONMENT STRATEGY

ENVIRONMENT	BRANCH	APPROVAL	PURPOSE
Development	feature/*	Optional	Test changes early
Staging	develop	Required	Validate before production
Production	main	Required (multiple reviewers)	Deploy to live environment

5. ARTIFACTS AND OUTPUTS

ARTIFACT	DESCRIPTION
tfplan	Saved plan file for review and apply
Plan logs	Pipeline logs for plan step
Apply logs	Pipeline logs for apply step
Outputs	Environment outputs (e.g., URLs, IDs)
Version info	Terraform and provider versions used
Metadata	Commit ID, author, timestamp, environment

6. BEST PRACTICES

- ✓ Always use remote state with locking
- ✓ Use separate pipelines per environment
- ✓ Enforce manual approval for production
- ✓ Run security and policy checks (tfsec, checkov, OPA)
- ✓ Store plan and apply artifacts
- ✓ Use least privilege credentials (OIDC or short-lived)
- ✓ Keep pipelines simple and reliable
- ✓ Monitor and alert on pipeline failures

7. COMMON MISTAKES

- ✗ Running apply without review
- ✗ Using the same credentials for all environments
- ✗ Not storing plan files and logs
- ✗ Skipping manual approval for production
- ✗ Allowing direct changes to main branch
- ✗ Not using policy or security checks
- ✗ Not handling failed or partial applies
- ✗ No audit trail or artifact retention

8. SENIOR ENGINEER TIP



"Your pipeline is your safety net. A good Terraform pipeline prevents mistakes today and protects your infrastructure tomorrow."



Instagram
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Telegram
veriqta



Medium
veriqta

BUILD
A BETTER TOMORROW

SECURE TERRAFORM AUTHENTICATION

Workload identity, OIDC, temporary credentials and least privilege

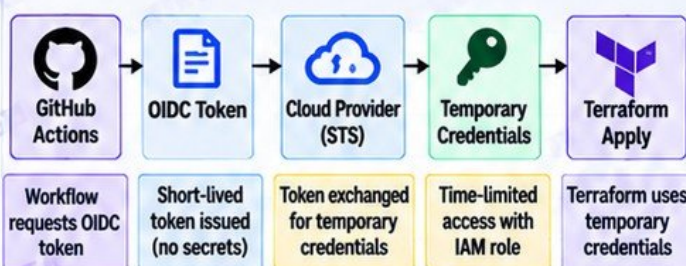
1. WHY SECURE AUTHENTICATION MATTERS

- ✓ Prevents long-lived and hardcoded credentials
- ✓ Reduces the risk of credential leaks
- ✓ Enables workload identity and short-lived access
- ✓ Supports least privilege and fine-grained access
- ✓ Improves compliance and auditability
- ✓ Protects production environments
- ✓ Minimizes blast radius if credentials are compromised

2. AUTHENTICATION METHODS OVERVIEW

METHOD	DESCRIPTION	USE CASE
Static credentials	Access key / secret key stored locally	Not recommended (for reference only)
Environment variables	Credentials from environment	Simple setups (non-production)
Shared credential files	Uses ~/.aws/credentials or cloud equivalents	Local development
OIDC / Workload identity	Short-lived credentials via identity provider	CI/CD, Kubernetes, cloud automation
Instance roles	Uses cloud instance metadata	EC2, VM, compute workloads
Federated access	Assume role via OIDC/SAML	Multi-account, enterprise
Terraform Cloud	Uses workspace identity	Managed Terraform runs

3. OIDC AUTHENTICATION FLOW (CI/CD EXAMPLE)



4. EXAMPLE: AWS OIDC WITH GITHUB ACTIONS

```
name: Terraform
on:
```

```
  push:
    branches: [ "main" ]
```

```
permissions:
```

```
  id-token: write
```

```
  contents: read
```

```
jobs:
```

```
  terraform:
```

```
    runs-on: ubuntu-latest
```

```
    steps:
```

```
      - name: Configure AWS credentials
```

```
        uses: aws-actions/configure-aws-credentials@v4
```

```
        with:
```

```
          role-to-assume: arn:aws:iam::123456789012:role/terraform
```

```
          aws-region: us-east-1
```

```
      - name: Terraform Plan
```

```
        run: terraform plan
```

KEY POINTS

- ✓ Uses OIDC (no static keys)
- ✓ Role with least privilege
- ✓ Short-lived credentials
- ✓ Works for any cloud provider with OIDC support
- ✓ Ideal for CI/CD pipelines

5. IAM POLICY EXAMPLE (LEAST PRIVILEGE)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket",
        "s3:GetObject",
        "dynamodb:DescribeTable"
      ],
      "Resource": [
        "arn:aws:s3:::my-terraform-state",
        "arn:aws:s3:::my-terraform-state/*",
        "arn:aws:dynamodb:us-east-1:123456789012:table/terraform-locks"
      ]
    }
  ]
}
```

BEST PRACTICES

- ✓ Allow only required actions
- ✓ Restrict resources
- ✓ Use conditions (e.g., source IP, OIDC claims)
- ✓ Use separate roles per environment
- ✓ Review and rotate regularly

6. COMMON MISTAKES

- ✗ Using long-lived access keys
- ✗ Hardcoding credentials in code
- ✗ Overly permissive IAM policies
- ✗ Not using OIDC or workload identity
- ✗ Sharing credentials across environments
- ✗ No separation of duties
- ✗ Not rotating credentials
- ✗ Ignoring audit logs
- ✗ Using the same role for all projects

7. SENIOR ENGINEER TIP



"Static credentials create permanent risk. Use workload identity, short-lived credentials and least privilege to keep your infrastructure safe."

SECRETS AND SENSITIVE DATA

Preventing credentials and sensitive infrastructure data from leaking through Terraform

1. WHY IT MATTERS

- ✓ Prevents credential and sensitive data leaks
- ✓ Protects your infrastructure and business from compromise
- ✓ Maintains compliance and security standards
- ✓ Reduces the risk of accidental exposure in code, state and logs
- ✓ Enables secure collaboration in teams
- ✓ Builds trust and operational safety

2. WHERE SENSITIVE DATA APPEARS

					
.tf	tfvars	State	Logs	Outputs	Providers
Configuration files	Variable files	Terraform state file	Plan and apply logs	Terraform outputs	Provider configuration



Sensitive data can leak in code, state files, logs, outputs and provider configuration if not handled correctly.

3. MARKING SENSITIVE VARIABLES

```
# Variable marked as sensitive
variable "db_password" {
  type      = string
  sensitive = true
  description = "Database password"
}
```

KEY POINTS

- ✓ Use sensitive = true for sensitive variables
- ✓ Prevents values from showing in plan and logs
- ✓ Still stored in state (encrypt state backend)
- ✓ Use secure input methods (TF_VAR_ or environment variables)

4. USING SECRETS MANAGERS (EXAMPLE: AWS SECRETS MANAGER)

```
data "aws_secretsmanager_secret_version" "db" {
  secret_id = "prod/db/password"
}

locals {
  db_password = jsondecode(
    data.aws_secretsmanager_secret_version.db.secret_string
  )["password"]
}

resource "aws_db_instance" "example" {
  identifier = "prod-db"
  password  = local.db_password
}
```

KEY POINTS

- ✓ Use cloud secrets managers (AWS, Azure, GCP, Vault)
- ✓ Do not hardcode secrets in configuration
- ✓ Use IAM roles / workload identity for access
- ✓ Secrets are retrieved at runtime, not stored in code

5. PROTECTING THE STATE FILE



- Use remote state backend (S3, Azure Blob, GCS, etc.)
- Enable encryption at rest (e.g., SSE-KMS)
- Restrict access with IAM policies
- Enable state locking (e.g., DynamoDB)
- Never use local state in production
- Regularly audit access to the state file

6. COMMON MISTAKES

- ✗ Hardcoding credentials in .tf files
- ✗ Storing secrets in .tfvars files
- ✗ Printing sensitive values in outputs
- ✗ Committing secrets to Git
- ✗ Using local state in production
- ✗ Not encrypting remote state
- ✗ Giving excessive permissions to tools or users
- ✗ Ignoring secrets in logs and CI/CD pipelines

7. BEST PRACTICES

- ✓ Use secrets managers (AWS Secrets Manager, Azure Key Vault, GCP Secret Manager)
- ✓ Mark sensitive variables and outputs
- ✓ Use environment variables or secure CI/CD secrets
- ✓ Encrypt and restrict access to remote state files
- ✓ Apply least privilege access (IAM roles, OIDC)
- ✓ Scan code and repositories for leaked secrets (e.g., GitHub secret scanning)
- ✓ Regularly rotate credentials and review access
- ✓ Educate your team on secure handling of sensitive data

8. SENIOR ENGINEER TIP



"Secrets are not just values, they are responsibilities. Treat every credential, key and token as production data. If it can be leaked, it will be leaked. Design your Terraform workflow so secrets never live in code, are encrypted in state, and are accessible only to those who need them."



Instagram
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Telegram
veriqta



Medium
veriqta

BUILD
A BETTER TOMORROW

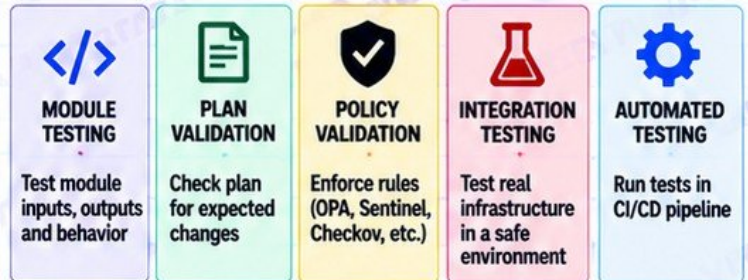
TERRAFORM TESTING AND VALIDATION

Testing modules, plans, policies and infrastructure changes before production

1. WHY TEST AND VALIDATE?

- ✓ Catches errors before production
- ✓ Reduces risk and outages
- ✓ Ensures infrastructure follows standards
- ✓ Validates security and compliance policies
- ✓ Improves code quality and reliability
- ✓ Prevents accidental destructive changes
- ✓ Gives confidence in your infrastructure changes
- ✓ Supports faster and safer delivery

2. TERRAFORM TESTING LAYERS



"Test early. Fail early. Deploy with confidence."

3. EXAMPLE: TERRATEST (GO)

```
package test

import (
    "testing"
    "github.com/gruntwork-io/terratest/modules/terraform"
)

func TestTerraformExample(t *testing.T) {
    terraformOptions := &terraform.Options {
        TerraformDir: "../examples/vpc",
    }
    defer terraform.Destroy(t, terraformOptions)
    terraform.InitAndApply(t, terraformOptions)
    output := terraform.Output(t, terraformOptions, "vpc_id")
    if output == "" {
        t.Fatal("vpc_id is empty")
    }
}
```

KEY POINTS

- ✓ Tests real Terraform code
- ✓ Validates outputs
- ✓ Catches issues early
- ✓ Can be integrated in CI/CD
- ✓ Works with any provider

4. POLICY AS CODE (OPA / CHECKOV EXAMPLE)

Example OPA policy (Rego)

```
package terraform.deny

deny[msg] {
    resource := input.resource_changes[_]
    resource.type == "aws_s3_bucket"
    not resource.change.after.server_side_encryption_configuration

    msg := "S3 bucket must have encryption enabled"
}
```

KEY POINTS

- ✓ Enforces security and compliance
- ✓ Prevents risky configurations
- ✓ Works with OPA, Sentinel or Checkov
- ✓ Integrates with CI/CD pipelines
- ✓ Custom rules for your organization

5. VALIDATION TOOLS COMPARISON

TOOL	TYPE	USE CASE	LANGUAGE
Terratest	Testing	Test modules and infrastructure	Go
Kitchen-Terraform	Testing	Integration testing with Test Kitchen	Ruby
Checkov	Policy	Security and compliance checks	Python
tfsec	Security	Static analysis for Terraform	Go
OPA (Open Policy Agent)	Policy	Custom policy rules	Rego
Terraform Validate	Built-in	Syntax and configuration check	N/A

6. BEST PRACTICES

- ✓ Test modules with different inputs and scenarios
- ✓ Validate plans before applying
- ✓ Use policy as code for security and compliance
- ✓ Run automated tests in CI/CD
- ✓ Check for destructive changes (use plan output)
- ✓ Test in isolated environments
- ✓ Keep tests simple, fast and reliable
- ✓ Continuously improve test coverage

7. COMMON MISTAKES

- ✗ Skipping tests and relying on manual checks
- ✗ Not validating plan output
- ✗ Ignoring security and compliance policies
- ✗ Only testing happy path scenarios
- ✗ Using production credentials in tests
- ✗ Not integrating tests into CI/CD
- ✗ Overly complex or slow tests
- ✗ Not testing module updates

8. SENIOR ENGINEER TIP



"A well-tested Terraform codebase is a force multiplier. It reduces incidents, builds trust and lets you move faster without fear."

9. REMEMBER



- ✓ Test
- ✓ Validate
- ✓ Enforce
- ✓ Automate
- ✓ Deploy with confidence

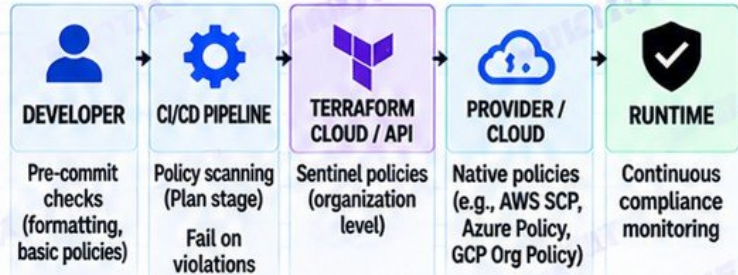
POLICY AS CODE AND GOVERNANCE

Guardrails, enforcement points, compliance and controlled exceptions

1. WHY IT MATTERS

- ✓ Enforces organizational standards
- ✓ Reduces security and compliance risks
- ✓ Prevents misconfigurations and drift
- ✓ Ensures consistent and auditable infrastructure
- ✓ Supports regulatory requirements (ISO, SOC2, etc.)
- ✓ Enables controlled exceptions with approval
- ✓ Improves governance and visibility
- ✓ Keeps environments secure, compliant and reliable

2. POLICY ENFORCEMENT POINTS



MULTIPLE LAYERS. STRONGER GOVERNANCE.

3. EXAMPLE: OPA (OPEN POLICY AGENT)

```
# Deny resources without required tags
package terraform.policies.tags

deny[msg] {
  resource := input.resource_changes[_]
  resource.type == "aws_instance"
  not resource.change.after.tags["Environment"]
  msg := sprintf("Resource %s must have an 'Environment' tag", [resource.address])
}
```

KEY POINTS

- ✓ Write policies in Rego language
- ✓ Validate Terraform plan input
- ✓ Block non-compliant resources
- ✓ Custom rules for your organization
- ✓ Integrates with CI/CD and Terraform Cloud

4. EXAMPLE: SENTINEL (TERRAFORM CLOUD)

```
import "tfplan/v2"

main = rule {
  all tfplan.resources as r {
    r.type is "aws_s3_bucket" ->
    r.values.encrypted_configuration
    is not null
  }
}

# Ensures all S3 buckets have encryption enabled
```

KEY POINTS

- ✓ Built into Terraform Cloud
- ✓ Simple and powerful policy language
- ✓ Enforce organization standards
- ✓ Manage policies centrally
- ✓ Support for exceptions with workspaces/teams

5. COMMON POLICIES

POLICY	PURPOSE
Enforce tagging	Require mandatory tags (Environment, Owner, etc.)
Encryption	Ensure storage and data are encrypted
Approved instance types	Restrict to allowed instance types
Region restrictions	Allow only approved regions
Network security	Enforce secure networking (no public access)
Resource limits	Prevent overly large or expensive resources
Compliance standards	Meet ISO, SOC2, PCI-DSS, etc.
Approved providers	Restrict to approved Terraform providers

6. BEST PRACTICES

- ✓ Define clear and practical policies
- ✓ Start with audit (warn) mode, then enforce
- ✓ Use multiple enforcement layers
- ✓ Keep policies version controlled (Git)
- ✓ Provide clear error messages and documentation
- ✓ Implement a controlled exception process
- ✓ Regularly review and update policies
- ✓ Align policies with business and compliance requirements
- ✓ Automate compliance reporting

7. COMMON MISTAKES

- ✗ Overly restrictive policies (blocks productivity)
- ✗ Weak or missing exception process
- ✗ Not testing policies before enforcement
- ✗ Ignoring provider-level controls (SCP, Azure Policy, etc.)
- ✗ Unclear or poorly documented policies
- ✗ Not keeping policies up to date

8. SENIOR ENGINEER TIP



"Governance is not about saying no. It is about enabling teams to move fast and safely with clear guardrails, visibility and trust."

9. REMEMBER



- ✓ Define
- ✓ Enforce
- ✓ Monitor
- ✓ Report
- ✓ Allow exceptions
- ✓ Keep improving



DRIFT DETECTION AND CONFIGURATION RECONCILIATION

Detecting, classifying and safely reconciling infrastructure drift

1. WHY IT MATTERS

- ✓ Detects unauthorized and unintended changes
- ✓ Keeps infrastructure consistent and predictable
- ✓ Reduces security and compliance risks
- ✓ Prevents configuration drift from accumulating
- ✓ Improves incident response and audit readiness
- ✓ Maintains trust in your infrastructure as code
- ✓ Ensures environments match the desired state
- ✓ Helps identify manual changes and external modifications

2. COMMON CAUSES OF DRIFT

 MANUAL CHANGES Changes made in cloud console or CLI	 AUTOMATION OUTSIDE TERRAFORM Scripts, tools or other IaC modifying resources	 THIRD-PARTY SERVICES Services that change resources automatically	 PROVIDER DEFAULTS Changes from provider updates or default values
 EXTERNAL SYSTEMS Auto-scaling, healing or managed services	 OUT-OF-BAND CHANGES Hotfixes and emergency changes	 INFRASTRUCTURE ADOPTION Resources created outside Terraform and later imported	 VERSION CHANGES Provider or service versions introducing differences

3. DETECTING DRIFT

```
# Check for drift
terraform plan

# Show detailed changes
terraform plan -detailed-exitcode

# Refresh state from real infrastructure
terraform plan -refresh-only

# Inspect current state
terraform state show aws_instance.web

# List all resources in state
terraform state list
```

KEY POINTS

- ✓ terraform plan shows differences between state and real infrastructure
- ✓ -detailed-exitcode returns 2 if changes are detected
- ✓ -refresh-only updates state without making changes
- ✓ Use state show and state list for investigation

4. DRIFT CLASSIFICATION

TYPE	DESCRIPTION	EXAMPLES	ACTION
Expected drift	Known and acceptable changes	Auto-scaling, AMI updates, timestamps	Document and ignore (if safe)
Unintentional drift	Changes not made through Terraform	Manual console through, emergency fixes	Reconcile back to desired state
Malicious drift	Unauthorized changes	Security group opened, IAM changes	Investigate and remediate immediately
Provider drift	Changes from provider behavior	Default settings, version updates	Review, update configuration if needed

5. RECONCILING DRIFT



Consistent infrastructure requires continuous drift detection and prompt reconciliation.

6. BEST PRACTICES

- ✓ Run regular drift detection (automated if possible)
- ✓ Use -detailed-exitcode in CI/CD
- ✓ Monitor for unauthorized changes
- ✓ Document and approve intentional changes
- ✓ Keep provider versions up to date
- ✓ Use tooling (e.g., Terraform Cloud, Spacelift, Infracost, etc.)
- ✓ Set up alerts for critical resources
- ✓ Regularly review and reconcile state
- ✓ Educate teams on the risks of manual changes

7. COMMON MISTAKES

- ✗ Ignoring drift warnings
- ✗ Relying only on manual checks
- ✗ Not investigating unknown changes
- ✗ Overlooking changes in critical resources
- ✗ Blindly applying changes without review
- ✗ Not keeping state files secure
- ✗ Assuming no drift in stable environments

8. SENIOR ENGINEER TIP



"Drift is inevitable. What separates a good engineer from a great one is how quickly they detect it, how well they understand it, and how safely they bring things back to the desired state."

9. REMEMBER



- ✓ Detect
- ✓ Classify
- ✓ Investigate
- ✓ Reconcile
- ✓ Validate
- ✓ Automate
- ✓ Stay vigilant

REFACTORING PRODUCTION INFRASTRUCTURE SAFELY

Moving and restructuring resources without unintended destruction

1. WHY REFACTOR?

- ✓ Improve code structure and maintainability
- ✓ Organize resources into modules
- ✓ Fix naming and tagging inconsistencies
- ✓ Move resources to different files or modules
- ✓ Adopt new standards and architectures
- ✓ Reduce technical debt
- ✓ Make infrastructure easier to manage
- ✓ Do all these without destroying live resources

2. COMMON REFACTORING SCENARIOS



MOVE RESOURCES

Move resources to new files or modules



RENAME RESOURCES

Change resource names without destruction



SPLIT CONFIGURATION

Split large files into smaller, logical modules



REORGANIZE MODULES

Restructure module layout and directories



UPDATE PROVIDERS

Upgrade provider versions and schemas



SAME INFRASTRUCTURE. BETTER STRUCTURE. NO UNINTENDED DESTRUCTION.

3. USING TERRAFORM STATE MV (MOVE RESOURCES)

```
# Move a resource to a new module/file
terraform state mv \
  aws_instance.web \
  module.compute.aws_instance.web

# Rename a resource address
terraform state mv \
  aws_s3_bucket.logs \
  aws_s3_bucket.app_logs

# Move from root module to module
terraform state mv \
  aws_security_group.sg \
  module.network.aws_security_group.sg
```

KEY POINTS

- ✓ Moves resource in state, not in cloud
- ✓ Prevents destruction and recreation
- ✓ Use exact resource addresses
- ✓ Verify with terraform plan after moving
- ✓ Commit state changes (if using remote backend)

4. REFACTORING WORKFLOW

1 PLAN THE CHANGE

Identify what to move, rename or restructure. Understand current state and dependencies.

2 BACKUP STATE

Create a state backup before making changes (especially for production).

3 MAKE CODE CHANGES

Reorganize files, modules or resource names.

4 UPDATE STATE

Use terraform state mv (or import if needed).

5 VALIDATE

Run terraform plan to ensure no unintended changes.

6 APPLY

Apply after successful validation.

7 MONITOR

Verify resources are unchanged and working as expected.

5. EXAMPLE: MOVE RESOURCE TO A MODULE

BEFORE (ROOT MODULE)

```
resource "aws_instance" "web" {
  ami      = "ami-12345678"
  instance_type = "t3.micro"
  tags = {
    Name = "web-server"
  }
}
```



AFTER (IN MODULE)

```
module "web" {
  source = "../modules/web"
}

# inside modules/web/main.tf
resource "aws_instance" "web" {
  ami      = "ami-12345678"
  instance_type = "t3.micro"
  tags = {
    Name = "web-server"
  }
}
```

STEPS

- 1 Move code to module
- 2 Run `terraform state mv aws_instance.web \ module.web.aws_instance.web`
- 3 Run `terraform plan` (should show no changes)
- 4 Apply and verify

6. BEST PRACTICES

- 1 Always backup your state
- ✓ Use terraform state mv for moves and renames
- ✓ Test changes in a non-production environment first
- ✓ Keep changes small and reviewed
- ✓ Use version control for all changes
- ✓ Run terraform plan and verify no destruction
- ✓ Document the changes and rationale
- ✓ Align with team and change management process
- ✓ Monitor resources after refactoring

7. COMMON MISTAKES

- ✗ Moving or renaming resources without using state mv
- ✗ Deleting and recreating resources accidentally
- ✗ Making too many changes at once
- ✗ Not backing up the state file
- ✗ Ignoring dependencies
- ✗ Not validating with terraform plan
- ✗ Refactoring in production without review
- ✗ Losing track of resource addresses

8. SENIOR ENGINEER TIP



"Refactoring is a sign of a healthy infrastructure codebase. You are not changing what it does, you are improving how it is structured. Do it carefully, do it incrementally, and always protect the state."

9. REMEMBER



- ✓ Plan
- ✓ Backup
- ✓ Move (state)
- ✓ Validate
- ✓ Apply
- ✓ Monitor
- ✓ Document



Instagram
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Telegram
veriqta



Medium
veriqta

BUILD
A BETTER TOMORROW

TERRAFORM PERFORMANCE AT SCALE

State size, graph complexity, provider throttling, parallelism and large estates

1. WHY PERFORMANCE MATTERS

- ✓ Faster and more reliable deployments
- ✓ Reduces CI/CD pipeline time and cost
- ✓ Avoids provider rate limiting and throttling
- ✓ Improves stability for large estates
- ✓ Helps teams scale Terraform usage safely
- ✓ Minimizes failures and timeouts
- ✓ Better developer experience and productivity
- ✓ Ensures consistent and predictable operations

2. KEY FACTORS AFFECTING PERFORMANCE

 STATE SIZE Large state files increase memory and processing time	 GRAPH COMPLEXITY More resources and dependencies increase plan and apply time	 PROVIDER THROTTLING Cloud providers have API rate limits (e.g., AWS, Azure, GCP)	 PARALLELISM Number of concurrent operations (--parallelism)	 LARGE ESTATES Multiple accounts, regions and environments increase scale and complexity
---	---	---	--	--



BIGGER INFRASTRUCTURE REQUIRES SMARTER DESIGN AND TUNING.

3. OPTIMIZING STATE SIZE

```
# Use remote state (S3 example)
terraform {
  backend "s3" {
    bucket = "veriqta-terraform-state"
    key    = "prod/terraform.tfstate"
    region = "us-east-1"
    encrypt = true
  }
}

# Keep state lean
# - Use separate state files
# - Avoid unnecessary resources
# - Use data sources instead of resources where possible
```

KEY POINTS

- ✓ Use remote backend (S3, Azure Blob, GCS, etc.)
- ✓ Split large infrastructure into multiple state files
- ✓ Remove unused resources from state
- ✓ Use data sources instead of managing existing resources
- ✓ Regularly audit and clean up state

4. HANDLING PROVIDER THROTTLING

```
# Example: Set retries in AWS provider
provider "aws" {
  region = "us-east-1"
  retry_mode = "standard"
  max_retries = 10
}

# Reduce parallelism if hitting rate limits
terraform apply -parallelism=5
```

KEY POINTS

- ✓ Understand provider API rate limits
- ✓ Use retries and backoff (built-in or custom)
- ✓ Reduce parallelism when needed
- ✓ Spread deployments across time/regions
- ✓ Monitor API usage and errors

5. PARALLELISM AND EXECUTION TIME

```
# Set parallelism (default is 10)
terraform apply -parallelism=20

# Or set in environment variable
export TF_PARALLELISM=20
```

KEY POINTS

- ✓ Increase parallelism for faster execution (if provider allows)
- ✓ Reduce parallelism when hitting rate limits
- ✓ Find the right balance for your environment
- ✓ Test and monitor for stability

6. SCALING FOR LARGE ESTATES



KEY POINTS

- ✓ Use multi-account and multi-region design
- ✓ Organize with modules and workspaces
- ✓ Use Terragrunt or orchestration tools for large estates
- ✓ Implement consistent naming and tagging
- ✓ Monitor performance and costs

7. COMMON MISTAKES

- ✗ Single large state file for everything
- ✗ Too many resources in one plan
- ✗ Ignoring provider rate limits
- ✗ Using default parallelism for large estates
- ✗ Not monitoring execution time and failures
- ✗ Overusing data sources or external lookups
- ✗ Not breaking infrastructure into smaller modules
- ✗ Ignoring state cleanup and optimization

8. SENIOR ENGINEER TIP



"Performance is not just about running Terraform faster. It is about designing a scalable, maintainable infrastructure codebase that can grow for years without becoming a bottleneck."

9. REMEMBER



- ✓ Optimize state size
- ✓ Manage graph complexity
- ✓ Handle provider throttling
- ✓ Tune parallelism
- ✓ Design for large estates
- ✓ Monitor and improve
- ✓ Keep it simple and modular

PRODUCTION FAILURE INVESTIGATION

Evidence-driven troubleshooting across Terraform, providers, state and cloud APIs

1. WHY A STRUCTURED INVESTIGATION MATTERS

- ✓ Reduces mean time to recovery (MTTR)
- ✓ Finds the real root cause, not symptoms
- ✓ Prevents repeat incidents
- ✓ Minimizes impact on production workloads
- ✓ Builds a culture of evidence and learning
- ✓ Helps differentiate Terraform, provider and cloud issues
- ✓ Improves reliability and operational confidence

2. COMMON FAILURE AREAS



TERRAFORM CONFIGURATION

- Syntax errors
- Invalid variables
- Wrong resource configuration
- Module issues



STATE AND BACKEND

- State lock issues
- State corruption
- Wrong workspace
- Backend connectivity
- Permission errors



PROVIDER ISSUES

- API rate limits
- Provider bugs
- Version incompatibility
- Authentication failures



CLOUD APIS AND SERVICES

- Service outages
- API errors
- Quota limits
- Throttling
- Regional issues



"Different layers. Different causes. One investigation."

3. INVESTIGATION WORKFLOW

1 IDENTIFY	Understand the problem and impact
2 COLLECT	Gather logs, error messages and evidence
3 ISOLATE	Determine which layer is failing
4 ANALYZE	Find the root cause using evidence
5 MITIGATE	Apply a safe fix or workaround
6 VERIFY	Confirm recovery and no side effects
7 DOCUMENT	Record findings and preventive actions

4. USEFUL COMMANDS FOR INVESTIGATION

```
# Enable detailed logs
export TF_LOG=DEBUG
export TF_LOG_PATH=terraform.log

# Reproduce the error
terraform init -reconfigure
terraform plan

# Check current state
terraform state list
terraform state show <resource_address>

# Check provider versions
terraform providers
terraform version
```

KEY POINTS

- ✓ Enable debug logs for detailed errors
- ✓ Reproduce the issue in a clean environment
- ✓ Check state and provider versions
- ✓ Compare with recent changes
- ✓ Use logs from cloud provider (AWS, Azure, GCP)
- ✓ Look for API error codes and request IDs

5. EXAMPLE ERROR ANALYSIS

ERROR MESSAGE	LIKELY CAUSE	HOW TO INVESTIGATE	FIX
Error acquiring the state lock	State lock exists	Check backend lock (e.g. S3 DynamoDB)	Release lock or wait for timeout
ThrottlingException	API rate limit	Check provider logs and CloudWatch	Reduce parallelism, retry with backoff
InvalidParameterValue	Wrong configuration	Review resource arguments	Fix configuration and reapply
Provider produced inconsistent result	Provider bug or API issue	Check provider version and GitHub issues	Upgrade provider or apply workaround
Resource not found	Manual change or drift	Check state vs cloud console	Import resource or recreate if needed

6. BEST PRACTICES

- ✓ Always collect and keep logs
- ✓ Investigate systematically (don't guess)
- ✓ Check recent changes (code, provider, cloud)
- ✓ Use isolated environment to reproduce
- ✓ Understand provider and cloud API error codes
- ✓ Monitor provider and API status pages
- ✓ Implement retry and backoff for transient errors
- ✓ Keep documentation of incidents and fixes
- ✓ Share learnings with your team
- ✓ Add preventive controls (alerts, policies, tests)

7. COMMON MISTAKES

- ✗ Ignoring error messages
- ✗ Assuming it's a Terraform bug
- ✗ Not checking provider or cloud status
- ✗ Making multiple changes at once
- ✗ Not reviewing recent code or manual changes
- ✗ Skipping logs and evidence
- ✗ Reapplying without understanding the cause
- ✗ Not documenting the incident

8. SENIOR ENGINEER TIP



"A production incident is not a failure, it is data. The faster you collect evidence, the faster you recover, and the stronger your infrastructure becomes."

9. REMEMBER



- ✓ Investigate
- ✓ Find the root cause
- ✓ Fix safely
- ✓ Verify
- ✓ Document
- ✓ Prevent

FAILED APPLY, RECOVERY AND DISASTER SCENARIOS

Partial applies, rollback limitations, state recovery and infrastructure restoration

1. WHY FAILED APPLIES HAPPEN

- ✓ Provider API errors and rate limits
- ✓ Network timeouts and transient failures
- ✓ Resource dependencies and ordering issues
- ✓ Invalid configuration or changed defaults
- ✓ Partial applies due to interrupted runs
- ✓ External changes (manual updates, policies, etc.)
- ✓ Bugs in providers or Terraform core
- ✓ Insufficient permissions or quotas

2. TYPES OF FAILURE SCENARIOS

 PARTIAL APPLY Some resources created, others failed	 PROVIDER FAILURES API errors, rate limits, timeouts	 STATE CORRUPTION State file inconsistent or damaged	 EXTERNAL CHANGES Resources modified outside Terraform	 DISASTER SCENARIOS Accidental deletion, region outage, major service failure
--	---	--	--	---

 NOT EVERY FAILURE CAN BE ROLLED BACK. PREPARATION MATTERS.

3. INVESTIGATION STEPS

- 1 Check the error message and identify the failing resource
- 2 Review Terraform logs (enable TF_LOG if needed)
- 3 Check provider and cloud service status
- 4 Verify the current state (terraform state list)
- 6 Compare with actual infrastructure in the cloud
- 6 Identify what was created, changed or not created
- 7 Determine the root cause and impact (blast radius)

4. RECOVERY OPTIONS

SCENARIO	RECOVERY APPROACH	KEY COMMANDS / ACTIONS
Partial apply	Complete the apply or fix and re-run	terraform plan terraform apply
Failed resource creation	Fix configuration and re-apply	terraform apply (target if needed)
Resource stuck in state	Remove from state and re-create	terraform state rm <resource> terraform import (if exists)
State corruption	Restore from backup or recover manually	Use remote state versioning or state pull/push
Accidental deletion	Recreate resource and import if possible	terraform import <resource> terraform plan
Major outage	Use backups, multi-region or recreate infrastructure	Follow disaster recovery plan and IaC configuration

5. USEFUL COMMANDS

```
# Enable detailed logs
export TF_LOG=DEBUG

# Check current state
terraform state list

# Show a specific resource
terraform state show aws_instance.web

# Remove a resource from state
terraform state rm aws_instance.web

# Import an existing resource
terraform import aws_instance.web i-1234567890abcdef0
```

KEY POINTS

- ✓ Use logs to understand the exact failure
- ✓ Verify state vs actual infrastructure
- ✓ Remove or import resources when needed
- ✓ Be careful with manual changes
- ✓ Always have state backups
- ✓ Document the recovery process

6. BEST PRACTICES

- ✓ Use remote state with versioning (S3, Azure, GCS)
- ✓ Enable state locking to prevent concurrent runs
- ✓ Validate changes with plan before apply
- ✓ Use smaller, incremental changes
- ✓ Set reasonable timeouts and retry logic
- ✓ Monitor provider and cloud service status
- ✓ Have a disaster recovery plan and infrastructure backups
- ✓ Automate recovery procedures where possible
- ✓ Document incidents, root causes and lessons learned
- ✓ Test recovery procedures in non-production environments

7. COMMON MISTAKES

- ✗ Assuming Terraform will automatically roll back
- ✗ Ignoring partial applies and inconsistent state
- ✗ Manually changing resources without updating state
- ✗ Not keeping state backups
- ✗ Relying on just one region or environment
- ✗ Skipping error log analysis
- ✗ Re-applying without understanding the root cause
- ✗ Not documenting the recovery steps

8. SENIOR ENGINEER TIP



"Disasters don't wait for the right time to happen. A reliable engineer is not the one who never fails, but the one who can recover quickly, keep calm, and restore services with confidence."

9. REMEMBER



- ✓ Investigate
- ✓ Understand
- ✓ Recover
- ✓ Validate
- ✓ Document
- ✓ Prevent
- ✓ Be prepared

PRODUCTION TERRAFORM TEAM OPERATING MODEL

Ownership, reviews, approvals, emergency access and infrastructure governance

1. WHY AN OPERATING MODEL MATTERS

- ✓ Defines clear ownership and accountability
- ✓ Reduces risk and prevents accidental changes
- ✓ Ensures consistent standards and best practices
- ✓ Enables safe and fast delivery at scale
- ✓ Improves security, compliance and audit readiness
- ✓ Supports incident response and emergency access
- ✓ Helps teams work efficiently and avoid conflicts
- ✓ Creates a culture of trust, collaboration and reliability

2. KEY ELEMENTS OF A TERRAFORM OPERATING MODEL



3. TEAM ROLES AND RESPONSIBILITIES

ROLE	KEY RESPONSIBILITIES
Terraform Owner	Owns modules and resources, ensures standards and documentation
Platform / DevOps Engineer	Maintains CI/CD, tooling and environments
Reviewer	Reviews code, plans and approves changes
Approver	Gives final approval for production changes
Security Team	Ensures security, compliance and access controls
Operations / SRE	Monitors, handles incidents and supports recovery
Leadership	Provides governance, sets policies and resolves escalations

4. CHANGE FLOW IN A PRODUCTION ENVIRONMENT



SAME PROCESS. SAFER CHANGES. MORE RELIABLE INFRASTRUCTURE.

5. EMERGENCY ACCESS (BREAK GLASS)



- ✓ Restrict to authorized personnel only
- ✓ Use time-limited and audited access
- ✓ Require approval and justification
- ✓ Follow a documented procedure
- ✓ Revoke access immediately after use
- ✓ Review and learn from every emergency access event

6. BEST PRACTICES

- ✓ Define clear ownership for every Terraform module and environment
- ✓ Use pull requests, code reviews and plan reviews
- ✓ Enforce approvals for production changes
- ✓ Implement least privilege access and secure credential management
- ✓ Maintain consistent standards, naming and module structure
- ✓ Use CI/CD pipelines with automated checks and policies
- ✓ Document procedures for normal and emergency changes
- ✓ Monitor and audit all changes and access
- ✓ Regularly review and improve the operating model

7. COMMON MISTAKES

- ✗ No clear ownership or accountability
- ✗ Skipping reviews or approvals
- ✗ Overly broad access permissions
- ✗ No emergency access procedure
- ✗ Inconsistent standards and module structure
- ✗ Poor communication between teams
- ✗ Not documenting changes and decisions
- ✗ Not learning from incidents

8. SENIOR ENGINEER TIP



"A strong operating model turns Terraform from a tool into a reliable, scalable and secure platform. It is not just about writing code, it is about how your team works, decides and operates together."

9. REMEMBER



- ✓ People
- ✓ Process
- ✓ Policy
- ✓ Security
- ✓ Communication
- ✓ Documentation
- ✓ Continuous improvement



Instagram
veriqta



GitHub
veriqta



LinkedIn
veriqta



X
veriqta



Telegram
veriqta



Medium
veriqta

BUILD
A BETTER TOMORROW

PRODUCTION READINESS AND SENIOR ENGINEER CHECKLIST

Final architecture, security, reliability, recovery and operational review

1. WHY PRODUCTION READINESS MATTERS

- ✓ Reduces risk of failures and outages
- ✓ Ensures secure and compliant infrastructure
- ✓ Improves reliability and recovery capability
- ✓ Validates architecture, design and configurations
- ✓ Builds operational confidence
- ✓ Prevents costly mistakes in production
- ✓ Demonstrates engineering maturity
- ✓ Helps teams deliver and maintain stable environments

2. KEY AREAS FOR PRODUCTION READINESS

 ARCHITECTURE Scalable, resilient and well designed	 SECURITY Access, secrets, compliance and policies	 RELIABILITY High availability and fault tolerance	 RECOVERY Backups, DR and tested restores	 OPERATIONS Monitoring, alerting and runbooks
 PEOPLE Clear ownership and on-call	 PROCESSES Reviews, approvals and change control	 TOOLING Validated providers, modules and pipelines	 GOVERNANCE Standards, tagging and cost control	 COST Right sizing and budget controls

3. PRODUCTION READINESS CHECKLIST

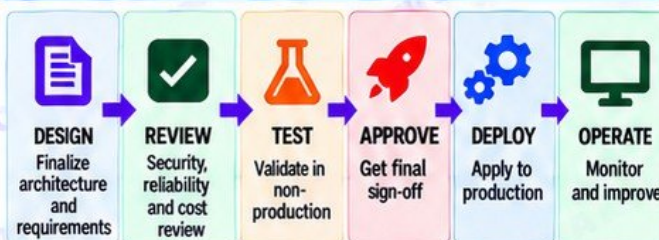
AREA	KEY CHECKS
Architecture	Modular, scalable, well documented, follows best practices
Security	Least privilege, secrets management, encryption, policy checks
Reliability	Multi-AZ/region, redundancy, failure handling
Recovery	Backups configured, tested restore process, DR plan
State Management	Remote backend, locking, versioning, access controls
Change Management	Code reviews, approvals, CI/CD, version control
Monitoring	Infrastructure and service monitoring, alerts, dashboards
Cost Management	Tagging, budgets, cost alerts, right sizing
Documentation	Architecture diagrams, runbooks, procedures
Team Readiness	Clear ownership, on-call, access management

 **PRODUCTION READY = SECURE + RELIABLE + RECOVERABLE + OPERATIONALLY SOUND**

4. SENIOR ENGINEER REVIEW QUESTIONS

- 1 Is the architecture scalable, resilient and cost efficient?
- 2 Are security controls, policies and compliance requirements met?
- 3 Are backups and disaster recovery tested and documented?
- 4 Are monitoring, alerting and logging in place?
- 5 Have the Terraform modules and configurations been reviewed?
- 6 Are there clear ownership and operational procedures?
- 7 Are failure scenarios and recovery steps documented and tested?
- 8 Is there a rollback or mitigation plan for changes?
- 9 Are all dependencies (providers, services, quotas) validated?
- 10 Is the team ready to operate and support this environment?

5. PRODUCTION READINESS FLOW



7. COMMON MISTAKES

- ✗ Skipping security and compliance checks
- ✗ Deploying without proper testing
- ✗ Poor state management and access control
- ✗ No backup or recovery plan
- ✗ Insufficient monitoring and alerting
- ✗ Lack of documentation
- ✗ Ignoring cost and resource limits
- ✗ Rushing to production without reviews
- ✗ No clear ownership or on-call process

8. SENIOR ENGINEER TIP

 "Production readiness is not a one-time checklist, it is a mindset. A senior engineer does not just deploy infrastructure, they ensure it can be operated, secured, recovered and trusted for the long term."

6. BEST PRACTICES

- ✓ Follow infrastructure as code best practices
- ✓ Use smallest necessary permissions
- ✓ Enable automated testing and policy checks
- ✓ Keep state secure and versioned
- ✓ Implement monitoring and alerting from day one
- ✓ Prepare and test disaster recovery procedures
- ✓ Document everything (architecture, runbooks, decisions)
- ✓ Conduct regular reviews and audits
- ✓ Involve the right stakeholders before production
- ✓ Continuously improve based on lessons learned

9. REMEMBER

- | | | |
|---|--|--|
|  | <ul style="list-style-type: none"> ✓ Plan ✓ Review ✓ Test ✓ Secure ✓ Document | <ul style="list-style-type: none"> ✓ Deploy ✓ Monitor ✓ Improve |
|---|--|--|